

Obtenção de covariáveis ambientais para ambientipagem

Gustavo Eduardo Marcatti - TheCROP / UFSJ

Este script demonstra como obter e processar covariáveis ambientais de fontes globais, como SoilGrids e NASA Power, para uma área de interesse definida a partir da localização de amostras contendo dados fenotípicos de genótipos, utilizando ferramentas disponíveis em Python.

Dados e Ferramentas

Dados

- **Link para download dos dados do curso:** [data_genotype.csv](#)
- **Link para download dos dados originais:** [DatasetS1.xlsx](#)
- **Publicação de origem:** <https://doi.org/10.1111/nph.19951>

Python

- **Distribuição recomendada:** [WinPython 3.12.10.1 \(64-bit, slim\)](#)
- **Página oficial:** <https://winpython.github.io/>

QGIS

- **Instalador:** [QGIS 3.44.11-1 \(Windows\)](#)
- **Site oficial:** <https://qgis.org/download/>

Dependências principais

```
pip install pandas geopandas rioxarray xarray geocube soilgrids rasterio matplotlib requests
```

Estrutura e etapas do código

1. Importação de bibliotecas

```
from pathlib import Path
import pandas as pd
import geopandas as gpd
from geocube.api.core import make_geocube
import rioxarray
import xarray as xr
from rasterio.enums import Resampling
from soilgrids import SoilGrids
import requests
import matplotlib.pyplot as plt
```

2. Criar área TPE (Target Population of Environment)

Leitura dos pontos genotípicos e geração de um polígono representando a área de estudo.

```

folder_work = Path(r"H:\Drive\curso\data")
table_genotype = pd.read_csv(folder_work / "data_genotype.csv",
                             decimal=".", sep=";")

points = gpd.points_from_xy(table_genotype.longitude, table_genotype.latitude)
point_genotype = gpd.GeoDataFrame(table_genotype,
                                   geometry=points, crs="EPSG:4326")

polygon_conc_hull = point_genotype.dissolve().concave_hull(ratio=0.5)
polygon_buff = polygon_conc_hull.buffer(distance=0.2)

```

Visualização da área:

```

fig, ax = plt.subplots()
polygon_buff.plot(ax=ax, color='green')
polygon_conc_hull.plot(ax=ax, color='blue')
point_genotype.plot(ax=ax, color='black', markersize=2)
fig

```

3. Criar grade espacial (raster e pontos)

O polígono é rasterizado e convertido em uma grade de pontos.

```

tpe_polygon = gpd.GeoDataFrame(geometry=polygon_buff)
tpe_polygon['constante'] = 1
tpe_polygon.set_crs('EPSG:4326', allow_override=True)
tpe_polygon.to_file(folder_work/"tpe_polygon.gpkg",
                    layer='tpe_polygon', driver="GPKG")

cube = make_geocube(
    tpe_polygon,
    measurements=["constante"],
    resolution=(0.01, 0.01)
)
cube.constante.rio.to_raster(folder_work / "tpe_raster.tif")

```

Conversão do raster em pontos:

```

tpe_raster = rioarray.open_rasterio(folder_work / "tpe_raster.tif")
coords_grid = tpe_raster.to_dataframe('value').reset_index()
coords_grid = (
    coords_grid
    .loc[(coords_grid['value'] == 1), ['x', 'y']]
    .reset_index(drop=True)
)
coords_grid['pixel'] = coords_grid.index + 1

tpe_point = gpd.GeoDataFrame(
    coords_grid.rename(columns={'x': 'longitude', 'y': 'latitude'}),
    geometry=gpd.points_from_xy(coords_grid.x, coords_grid.y),
    crs="EPSG:4326"
)
tpe_point.to_file(folder_work / "tpe_point.gpkg",
                  layer='tpe_point', driver="GPKG")

```

4. Download de covariáveis ambientais

SoilGrids

Obtém variáveis de solo (ex: areia, argila, carbono orgânico etc.) em diferentes profundidades.

```
coords_bounds = tpe_raster.rio.bounds()
lon_min, lat_min, lon_max, lat_max = coords_bounds
dim_width = tpe_raster.rio.width
dim_height = tpe_raster.rio.height

services = ['bdod', 'cec', 'cfvo', 'clay', 'nitrogen',
            'phh2o', 'sand', 'silt', 'soc', 'ocd']
depth = ['0-5cm', '5-15cm', '15-30cm', '30-60cm', '60-100cm', '100-200cm']
prediction = ['Q0.05', 'Q0.5', 'mean', 'Q0.95']
data_soil = [
    (i, f"{i}_{j}_{k}")
    for i in services
    for j in depth
    for k in prediction
]
data_soil.extend([( 'ocs', f'ocs_0-30cm_{k}') for k in prediction])
```

Loop de download:

```
soilgrids = SoilGrids()
folder_soilgrids = folder_work / 'soilgrids'
folder_soilgrids.mkdir(parents=True, exist_ok=True)

for service_id, coverage_id in data_soil:
    try:
        name_raster = f'soil_grids_{coverage_id}.tif'
        soilgrids.get_coverage_data(
            service_id=service_id,
            coverage_id=coverage_id,
            west=lon_min, south=lat_min,
            east=lon_max, north=lat_max,
            width=dim_width, height=dim_height,
            crs='urn:ogc:def:crs:EPSG::4326',
            output=str(folder_soilgrids / name_raster)
        )
        print(f'Download SoilGrids OK: {coverage_id}')
    except:
        print(f'Falhou: {coverage_id}')
```

NASA POWER

Coleta variáveis climáticas médias (temperatura, precipitação, vento, radiação etc.).

```
folder_nasapower = folder_work / 'nasapower'
folder_nasapower.mkdir(parents=True, exist_ok=True)

parameters = ['T2M', 'T2M_MAX', 'T2M_MIN', 'PRECTOTCORR', 'WS2M', 'RH2M',
              'T2MDEW', 'ALLSKY_SFC_LW_DWN', 'ALLSKY_SFC_SW_DWN']
```

```
url_nasa_power = 'https://power.larc.nasa.gov/api/temporal/climatology/regional?'
factor = 0.5 / 0.01 # 0.5° (NASA) → 0.01° (TPE)
expand = 0.2
```

Loop de download e reamostragem:

```
for parameter in parameters:
    url = url_nasa_power
    url += f'&latitude-min={lat_min-expand}&latitude-max={lat_max+expand}'
    url += f'&longitude-min={lon_min-expand}&longitude-max={lon_max+expand}'
    url += f'&community=ag&parameters={parameter}&format=xarray'
    response = requests.get(url)
    print(f'Download Nasa Power: {parameter}, status: {response.status_code}')

    nasa_power_ds = xr.Dataset.from_dict(response.json())
    nasa_power_ds = nasa_power_ds.rename({'lon': 'x', 'lat': 'y'})
    nasa_power_ds.rio.set_spatial_dims(x_dim='x', y_dim='y', inplace=True)
    nasa_power_ds.rio.write_crs('EPSG:4326', inplace=True)

    new_width = int(nasa_power_ds.rio.width * factor)
    new_height = int(nasa_power_ds.rio.height * factor)
    nasa_power_sampled = nasa_power_ds.rio.reproject(
        nasa_power_ds.rio.crs,
        shape=(new_height, new_width),
        resampling=Resampling.cubic_spline,
    )
    path_out = folder_nasapower / f'nasa_power_{parameter}.tif'
    nasa_power_sampled[parameter].rio.to_raster(path_out)
```

5. Extração de valores de covariáveis

Função para extrair valores de SoilGrids e NASA Power em coordenadas de pontos.

```
def get_cov(df_cov, tgt_x, tgt_y, name_csv):
    soil_path = list((folder_work / 'soilgrids').glob("soil_grids_*"))
    for path in soil_path:
        soil_r = rioxarray.open_rasterio(path)
        var_name = path.name.replace('soil_grids_', '').replace('.tif', '')
        var_array = soil_r.sel(x=tgt_x, y=tgt_y, method="nearest").values[0]
        var_pd = pd.DataFrame({var_name: var_array})
        df_cov = pd.concat([df_cov, var_pd], axis=1)

    nasa_path = list((folder_work / 'nasapower').glob("nasa_power_*"))
    for path in nasa_path:
        nasa_r = rioxarray.open_rasterio(path)
        var_name = path.name.replace('nasa_power_', '').replace('.tif', '').lower()
        var_array = nasa_r.sel(x=tgt_x, y=tgt_y, method="nearest")
        col_name = [f'{var_name}_{name}' for name in var_array['band'].values]
        var_pd = pd.DataFrame(var_array.values.T, columns=col_name)
        df_cov = pd.concat([df_cov, var_pd], axis=1)

    df_cov.to_csv(folder_work / f'{name_csv}.csv', sep=';', decimal=',', index=False)
```

Uso:

```
sample_cov = pd.read_csv(folder_work / "data_genotype.csv", decimal=".", sep=";")
tgt_x = xr.DataArray(sample_cov.longitude.values, dims="points")
tgt_y = xr.DataArray(sample_cov.latitude.values, dims="points")
get_cov(sample_cov, tgt_x, tgt_y, "cov_sample")

grid_cov = gpd.read_file(folder_work /
"tpe_point.gpkg")[['pixel', 'longitude', 'latitude']]
tgt_x = xr.DataArray(grid_cov.longitude.values, dims="points")
tgt_y = xr.DataArray(grid_cov.latitude.values, dims="points")
get_cov(grid_cov, tgt_x, tgt_y, "cov_grid")
```

Saídas esperadas

- tpe_polygon.gpkg — polígono da área de estudo
- tpe_raster.tif — raster binário da área TPE
- tpe_point.gpkg — grade regular de pontos
- soilgrids/*.tif — variáveis edáficas
- nasapower/*.tif — variáveis climáticas
- cov_sample.csv e cov_grid.csv — valores de covariáveis extraídos